CCIP503 Computer Networks & Security

Week – 4

Application Layer



Assist. Prof. Dr. İlktan AR

0.1 Course – Tentative Outline

Week	Date	Topic
1	28 Sept.	Computer Networks and the Internet
2	5 Oct.	Computer Networks and the Internet
3	12 Oct.	Application Layer
4	26 Oct.	Application Layer
5	2 Nov.	Transport Layer, Wireshark usage and examples
6	9 Nov.	**First Midterm Exam (15:00-17:00 Balat Salon & PC.Lab3)**
7	16 Nov.	Transport Layer
8	23 Nov.	The Network Layer: Data Plane
9	30 Nov.	The Network Layer: Control Plane
10	30 Nov.	The Link Layer and Local Area Networks
11	7 Dec.	**Second Midterm Exam (15:00-17:00 Balat Salon & PC.Lab3)**
12	14 Dec.	Security Technology and Principles
13	21 Dec.	Security Technology and Principles
14	28 Dec.	Security Technology and Principles, Talk for Final Exam
FN	???	**Final Exam**

Week 4 - Agenda

01 The Domain Name System - DNS

02 P2P Applications

03 Video Streaming and Content Distribution Networks

04 Next Week

1.1 Introduction

- Internet hosts can be identified by its **hostname**. Hostnames such as www.facebook.com, www.google.com, gaia.cs.umass.edu are mnemoni(tr: anımsatıcı, belletici) and are therefore appreciated by humans.

- Hostnames can consist of variable-length alphanumeric characters, they would be difficult to process by routers (more reasons later in course, network layer). For this reason and other reasons hosts are also identified by so-called **IP addresses**.

1.2 Introduction (cont.)

- An IP address looks like 121.7.106.83, where each period separates one of the bytes expressed in decimal notation from 0 to 255. An IP address is hierarchical because as we scan the address from left to right, we obtain more specific information about where the host is located on the Internet.

- Problem arises: How to map between IP address and hostname, and vice versa ?



1.3 Domain Name System (DNS)

- We need a directory service that translates hostnames to IP addresses. This is the main task of the Internet's **domain name system** (DNS).

- The DNS is (1) a distributed database implemented in a hierarchy of DNS servers, and (2) an application-layer protocol that allows hosts to query the distributed database.

- The DNS servers are often UNIX machines running the Berkeley Internet Name Domain (BIND) software.

1.4 Domain Name System (cont.)

- **The DNS protocol runs over UDP and uses port 53**.

- Like HTTP, FTP, and SMTP, the DNS protocol is an application-layer protocol.

- Consider what happens when a browser (that is, an HTTP client), running on some user's host, requests the URL *www.someschool.edu/index.html* . For the user's host to be able to send an HTTP request message to the Web server *www.someschool.edu* , the user's host must first obtain the IP address of *www.someschool.edu* .

1.5 DNS: Services, Structure

DNS services:

- hostname-to-IP-address translation
- host aliasing (tr: takma ad)
  - canonical, alias names
- mail server aliasing
- load distribution
  - replicated Web servers: many IP addresses correspond to one name

A hostname such as relay1.west-coast.enterprise.com could have, say, two **aliases** such as enterprise.com and www.enterprise.com. In this case, the hostname relay1.west-coast.enterprise.com is said to be a **canonical hostname**.

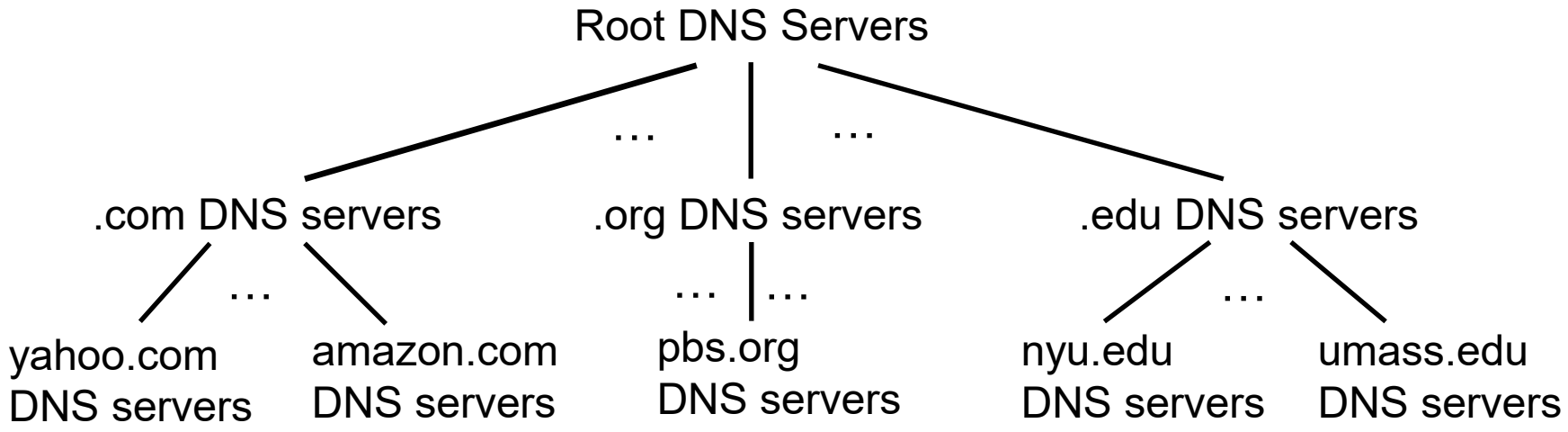
Q: Why not centralize DNS?

- single point of failure
- traffic volume
- distant centralized database
- maintenance

A: doesn't scale!

- Comcast DNS servers alone: 600B DNS queries/day
- Akamai DNS servers alone: 2.2T DNS queries/day

1.6 DNS: Distributed, Hierarchical Database



Root

Top Level Domain

Authoritative

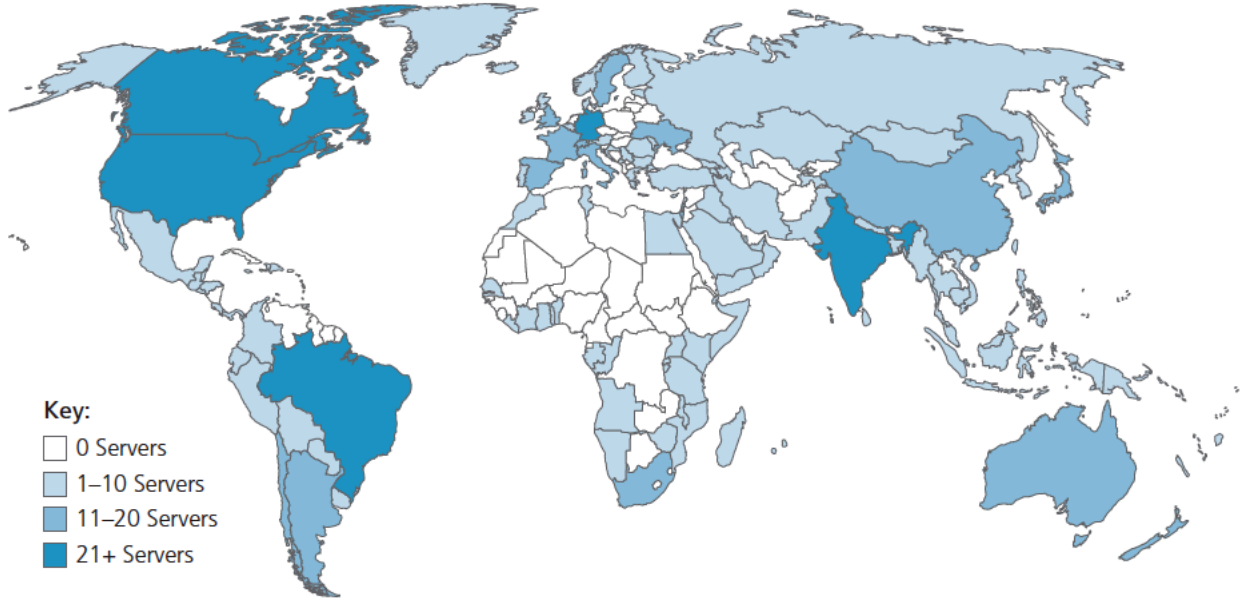
Client wants IP address for www.amazon.com; 1st approximation:

- client queries root server to find .com DNS server
- client queries .com DNS server to get amazon.com DNS server
- client queries amazon.com DNS server to get IP address for www.amazon.com

1.7 DNS: Root Name Servers

- official, contact-of-last-resort by name servers that can not resolve name
- *incredibly important* Internet function
  - Internet couldn't function without it!
  - DNSSEC – provides security (authentication, message integrity)
- ICANN (Internet Corporation for Assigned Names and Numbers) manages root DNS domain

13 logical root name "servers" worldwide each "server" replicated many times (~200 servers in US)

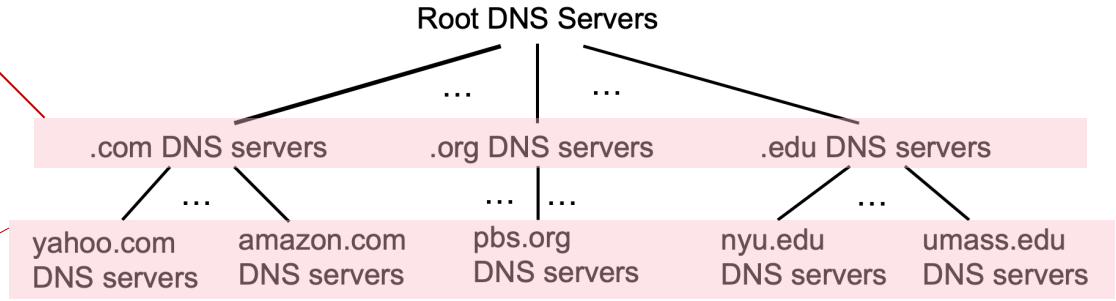


- Year=2016

1.8 Top-Level Domain, and Authoritative Servers

Top-Level Domain (TLD) servers:

- responsible for .com, .org, .net, .edu, .aero, .jobs, .museums, and all top-level country domains, e.g.: .cn, .uk, .fr, .ca, .jp
- Network Solutions: authoritative registry for .com, .net TLD
- Educause: .edu TLD



authoritative DNS servers:

- organization's own DNS server(s), providing authoritative hostname to IP mappings for organization's named hosts
- can be maintained by organization or service provider

1.9 Local DNS servers (aka Default Name Server)

- when host makes DNS query, it is sent to its *local* DNS server
  - Local DNS server returns reply, answering:
    - from its local cache of recent name-to-address translation pairs (possibly out of date!)
    - forwarding request into DNS hierarchy for resolution
  - each ISP has local DNS name server; to find yours:
    - MacOS: `% scutil --dns`
    - Windows: `>ipconfig /all`
- local DNS server doesn't strictly belong to hierarchy



1.10 From URL to First Byte: A DNS Walkthrough

1 · Browser cache

Has this hostname already been resolved recently?

2 · Stub resolver

Ask the configured recursive resolver; the application need not know the hierarchy.

3 · Recursive cache

A valid cached answer can finish the lookup immediately.

4 · DNS hierarchy

On a miss: root referral → TLD referral → authoritative answer.

5 · Connect

The client connects to the returned IP; DNS does not fetch the web page.

Important distinction

DNS returns records, not HTML. A browser can resolve a name successfully and still fail to connect to the service.

1.11 DNS Delegation: Who Is Authoritative?

Root servers

Know where top-level domains are delegated. A root referral usually points to TLD name servers; it normally does not contain the final host address.

TLD servers

For example, the .com zone identifies the authoritative name servers delegated for example.com.

Authoritative servers

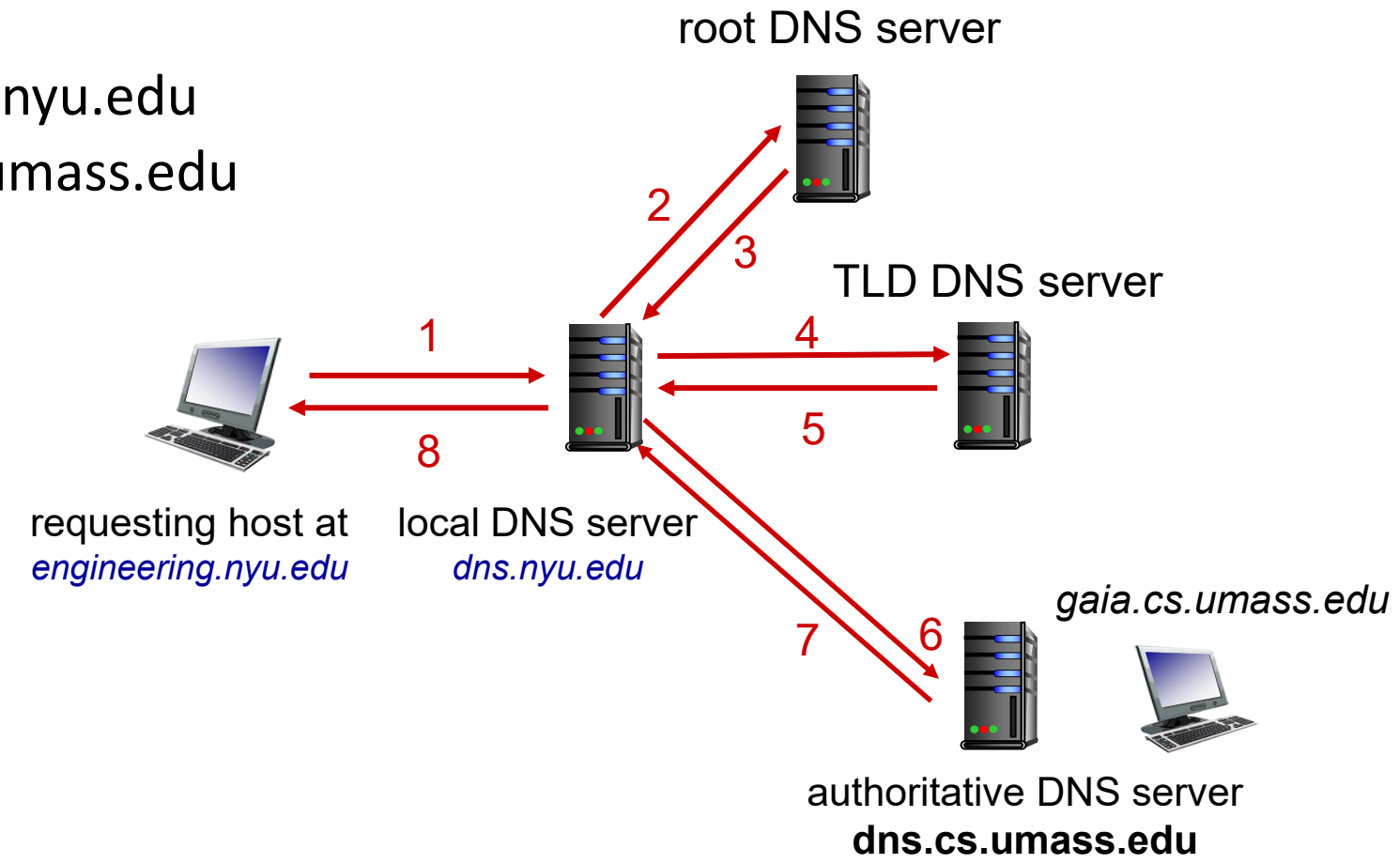
Return the records for names inside the zone, such as www.example.com. They may be operated by the organization or a DNS provider.

1.12 DNS Name Resolution: Iterated Query

Example: host at engineering.nyu.edu wants IP address for gaia.cs.umass.edu

Iterated query:

- contacted server replies with name of server to contact
- "I don't know this name, but ask this server"

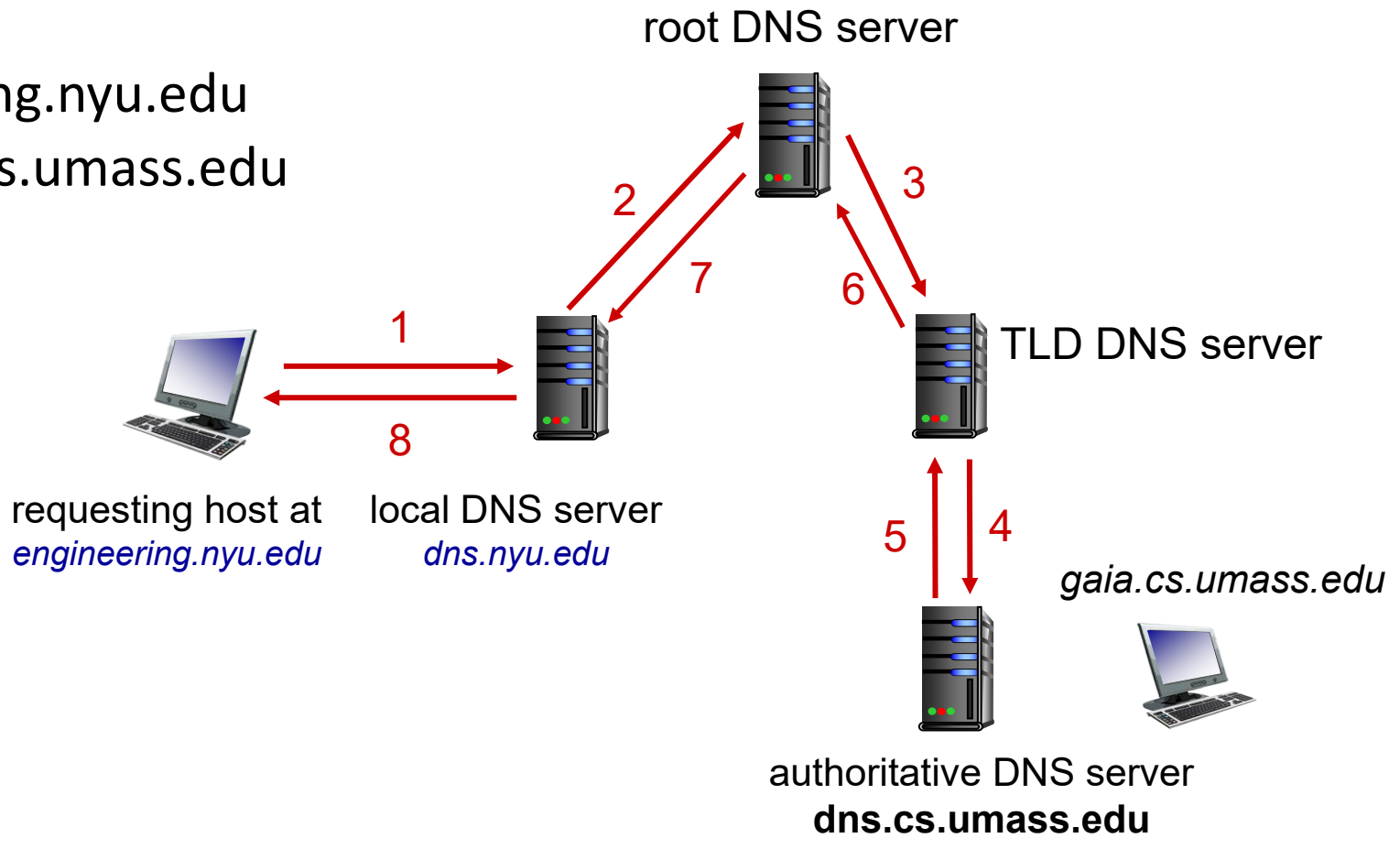


1.13 DNS Name Resolution: Recursive Query

Example: host at engineering.nyu.edu wants IP address for gaia.cs.umass.edu

Recursive query:

- puts burden of name resolution on contacted name server
- heavy load at upper levels of hierarchy?



1.14 DNS Caching

- DNS caching, a critically important feature of the DNS system.
- In truth, DNS extensively exploits DNS caching to improve the delay performance and to reduce the number of DNS messages.
- Once (any) name server learns mapping, it caches mapping, and immediately returns a cached mapping in response to a query
  - caching improves response time
  - cache entries timeout (disappear) after some time (TTL)
  - TLD servers typically cached in local name servers
- Cached entries may be out-of-date
  - if named host changes IP address, may not be known Internet-wide until all TTLs expire!
  - best-effort name-to-address translation!

1.15 DNS Records

DNS: distributed database storing resource records (RR)

RR format: (`name, value, type, ttl`)

type=A

- `name` is hostname
- `value` is IP address

(relay1.bar.foo.com,145.37.93.126, A)

type=NS

- `name` is domain (e.g., foo.com)
- `value` is hostname of authoritative name server for this domain (foo.com, dns.foo.com, NS)

type=CNAME

- `name` is alias name for some "canonical" (the real) name
- www.ibm.com is really servereast.backup2.ibm.com
- `value` is canonical name

type=MX

- `value` is name of SMTP mail server associated with `name`

TTL is the time to live of the resource record; it determines when a resource should be removed from a cache.

1.16 Resource Records: Beyond A and CNAME

A and AAAA

A maps a name to an IPv4 address. AAAA maps a name to an IPv6 address. A service can publish both.

CNAME and NS

CNAME aliases one name to another canonical name.
NS identifies the authoritative name servers for a zone/delegation.

shop.example.com. CNAME stores.example.net

The .com TLD zone utilizes NS (Name Server) records to indicate the authoritative DNS servers designated for example.com

MX and TXT

MX identifies mail exchangers and their preference.
TXT stores text data used for verification and email policies (e.g., SPF-related records).

example.com. MX 10 mail1.example.com.

PTR and SRV

PTR (Pointer) records are utilized in reverse DNS lookup to map an IP address to its associated hostname.

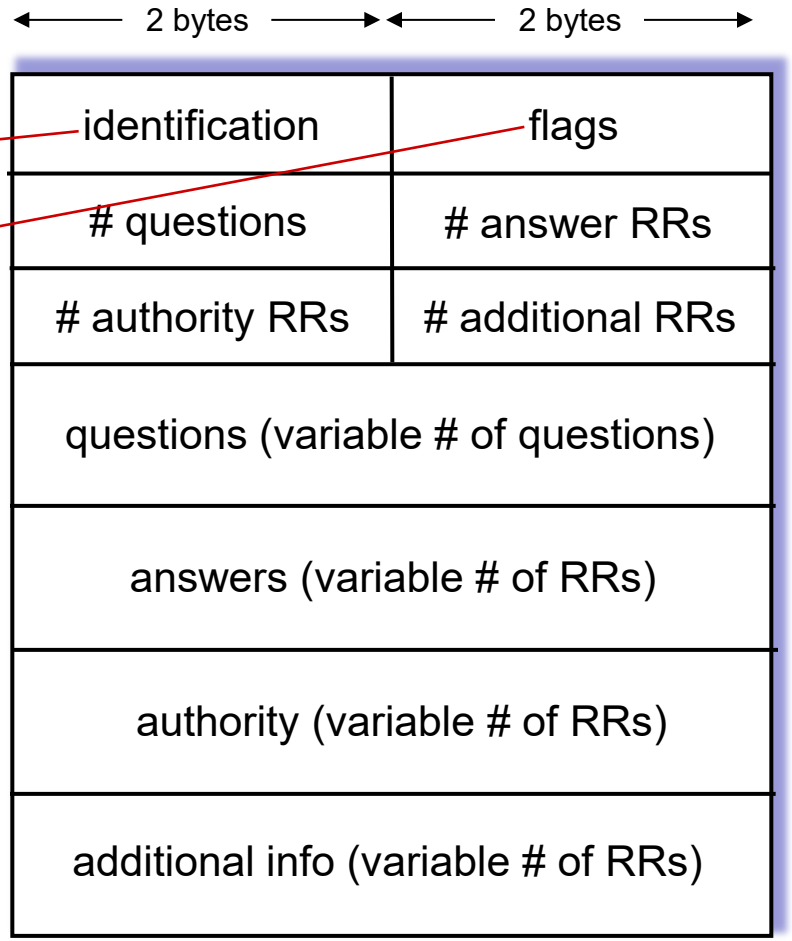
SRV (Service) records are used to advertise the hostname and port number on which a specific service is hosted.

1.17 DNS Protocol Messages

DNS *query* and *reply* messages, both have same *format:*

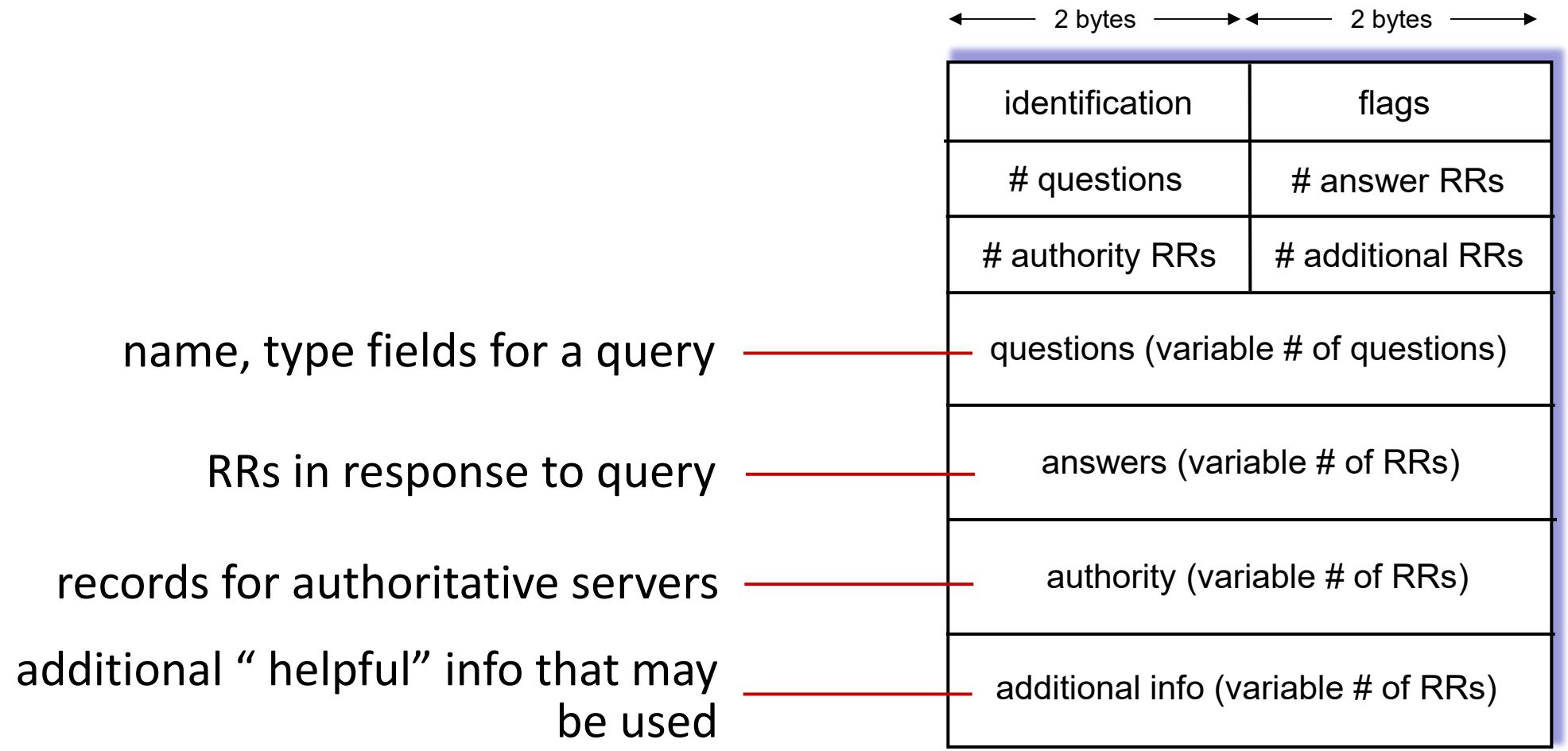
message header:

- identification: 16 bit # for query, reply to query uses same #
- flags:
  - query or reply
  - recursion desired
  - recursion available
  - reply is authoritative



1.18 DNS Protocol Messages (cont.)

DNS *query* and *reply* messages, both have same *format:*



1.19 Inserting Records into the DNS Database

example: new startup “Network Utopia”

- register name networkuptopia.com at *DNS registrar* (e.g., Network Solutions)
  - provide names, IP addresses of authoritative name server (primary and secondary)
  - registrar inserts NS, A RRs into .com TLD server:
    `(networkutopia.com, dns1.networkutopia.com, NS)`
    `(dns1.networkutopia.com, 212.212.212.1, A)`
- create authoritative server locally with IP address `212.212.212.1`
  - type A record for www.networkuptopia.com
  - type NS record for networkutopia.com

https://www.icann.org/en/accredited-registrars?page=1&country=Turkey&view-all=true

1.20 DNS Security: Four Real-World Failure Modes

Cache poisoning

An attacker tries to inject forged records into a resolver cache. Strong transaction matching, source-port randomization, and DNSSEC validation help defend against different aspects of this risk.

DNS hijacking

A resolver setting, registrar account, or authoritative zone is changed so users are directed to an attacker-controlled destination.

Typosquatting

A look-alike domain exploits typing mistakes or visual similarity. DNS can resolve it correctly even though the destination is untrustworthy.

Availability attacks

Flooding authoritative or recursive infrastructure can make names fail to resolve. Redundancy, rate limiting, monitoring, and resilient delegation matter.


10 mins break,
QR attendance!
TIME
FOR A
BREAK!

Week 4 - Agenda

01 The Domain Name System - DNS

02 P2P Applications

03 Video Streaming and Content Distribution Networks

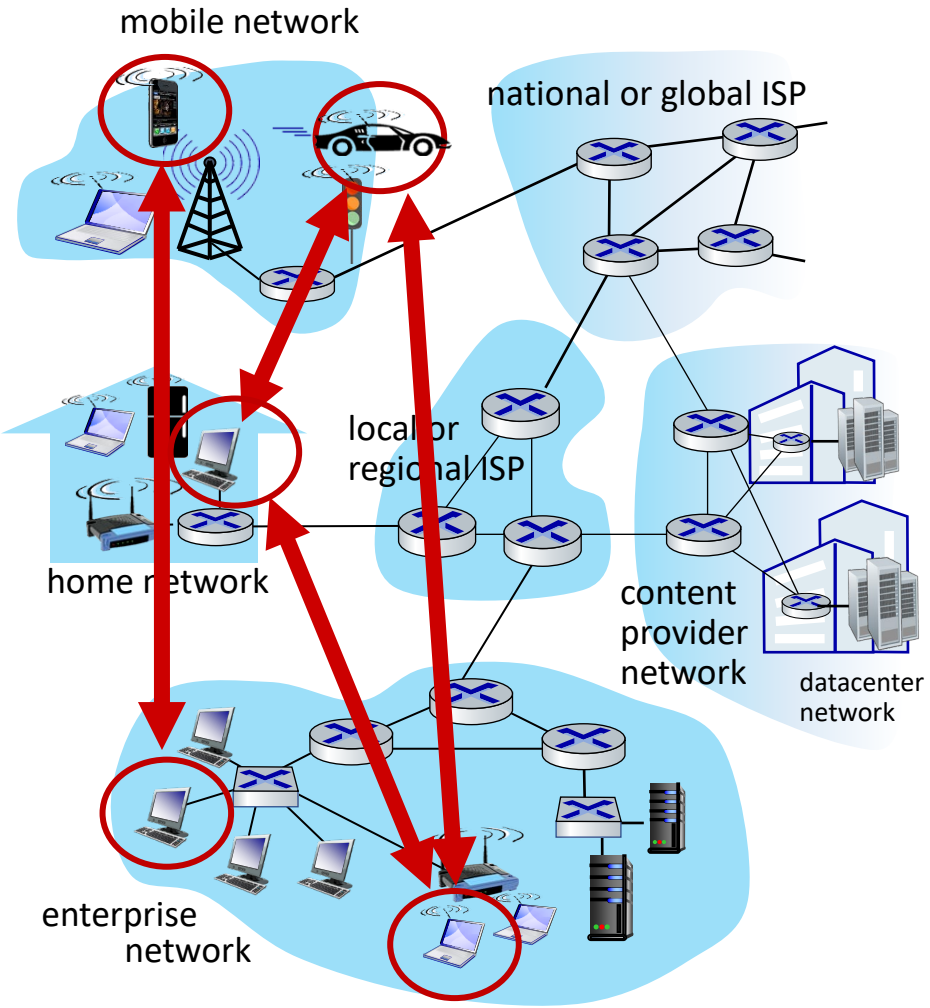
04 Next Week

2.1 Introduction

- The applications described in this course thus far including the Web, e-mail, and DNS all employ client-server architectures with significant reliance on always-on infrastructure servers.
- P2P architecture, there is minimal (or no) reliance on always-on infrastructure servers.
- As of 2016, the most popular P2P file distribution protocol is BitTorrent.
- Originally developed by Bram Cohen, there are now many different independent BitTorrent clients conforming to the BitTorrent protocol, just as there are several Web browser clients that conform to the HTTP protocol.

2.2 Briefly P2P Architecture

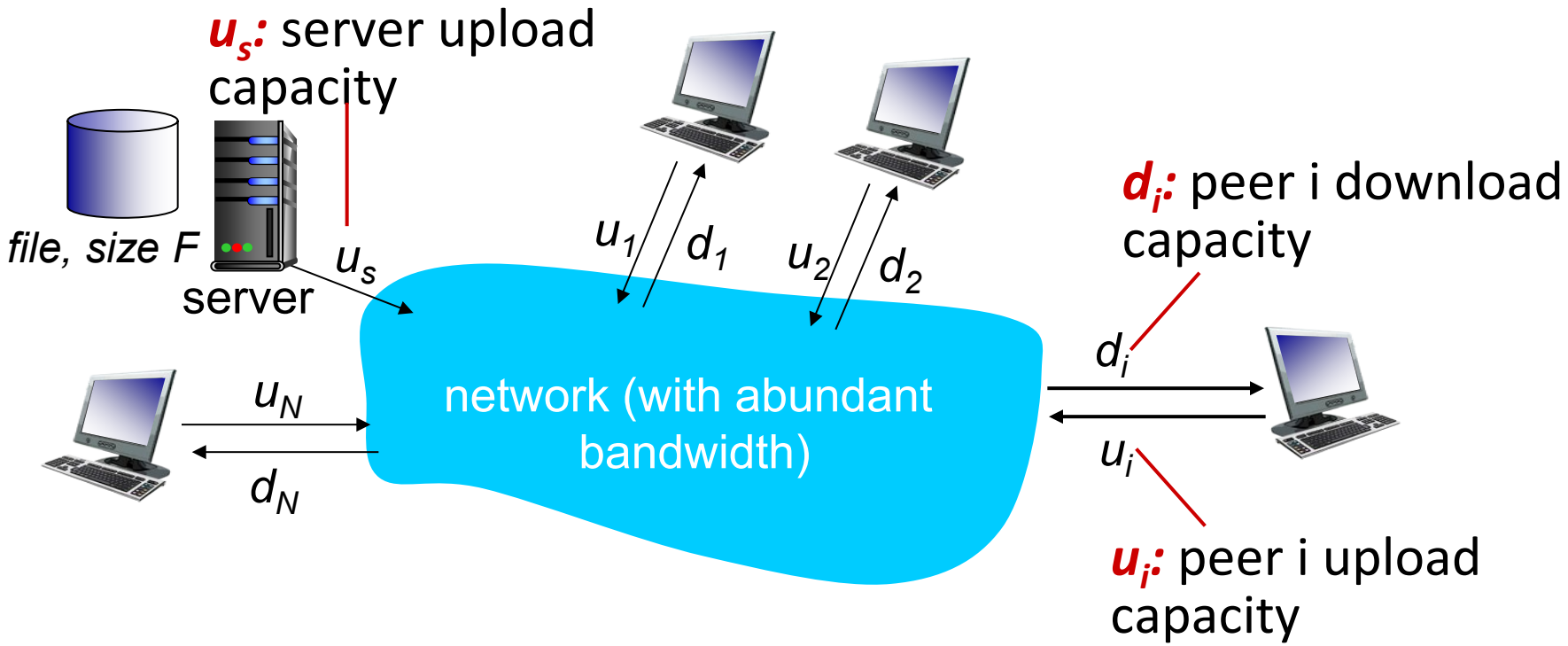
- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
  - *self scalability* – new peers bring new service capacity, and new service demands
- peers are intermittently connected and change IP addresses
  - complex management
- examples: P2P file sharing (BitTorrent), streaming (KanKan), VoIP (Skype)



2.3 File Distribution: Client-server vs P2P

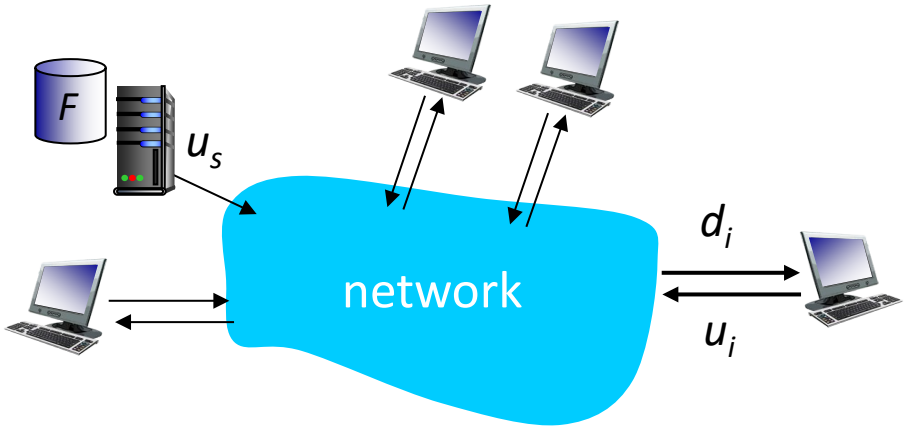
Q: how much time required to distribute file (size *F*) from one server to *N peers*?

- peer upload/download capacity is limited resource



2.4 File Distribution Time: Client-Server

- *server transmission:* must sequentially send (upload) *N* file copies:
  - time to send one copy: F/u_s
  - time to send *N* copies: NF/u_s
- *client:* each client must download file copy
  - d_{min} = min client download rate
  - min client download time: F/d_{min}



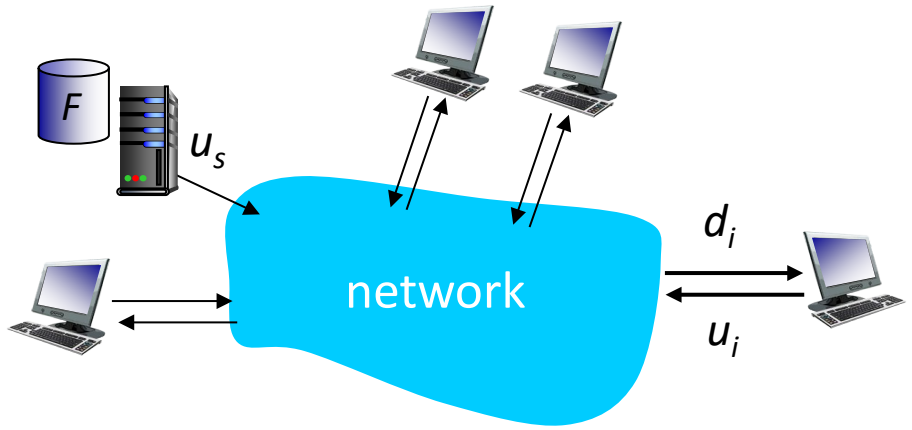
time to distribute F to N clients using client-server approach

$$D_{c\text{-}s} \geq max\{NF/u_s, F/d_{min}\}$$

increases linearly in N

2.5 File Distribution Time: P2P

- *server transmission:* must upload at least one copy:
  - time to send one copy: F/u_s
- *client:* each client must download file copy
  - min client download time: F/d_{min}
- *clients:* as aggregate must download NF bits
  - max upload rate (limiting max download rate) is $u_s + \Sigma u_i$



time to distribute F to N clients using P2P approach

$$D_{P2P} \geq max\{F/u_s, F/d_{min}, NF/(u_s + \Sigma u_i)\}$$

increases linearly in N …

… but so does this, as each peer brings service capacity

2.6 File Distribution Time: P2P (cont.)

- At the beginning of the distribution, only the server has the file. To get this file into the community of peers, the server must send each bit of the file at least once into its access link. Thus, the minimum distribution time is at least F/u . (Unlike the client-server scheme, a bit sent once by the server may not have to be sent by the server again, as the peers may redistribute the bit among themselves.)

- Observe that the total upload capacity of the system is equal to the upload rate of the server plus the upload rates of each of the individual peers, that is, $u_{total}=u_s+u_1+\cdots+u_N$.

- The system must deliver (upload) F bits to each of the N peers, thus delivering a total of NxF bits. This cannot be done at a rate faster than u_{total}.

2.7 Client-Server vs. P2P: Example

client upload rate = u, F/u = 1 hour, $u_s = 10u$, $d_{min} \geq u_s$

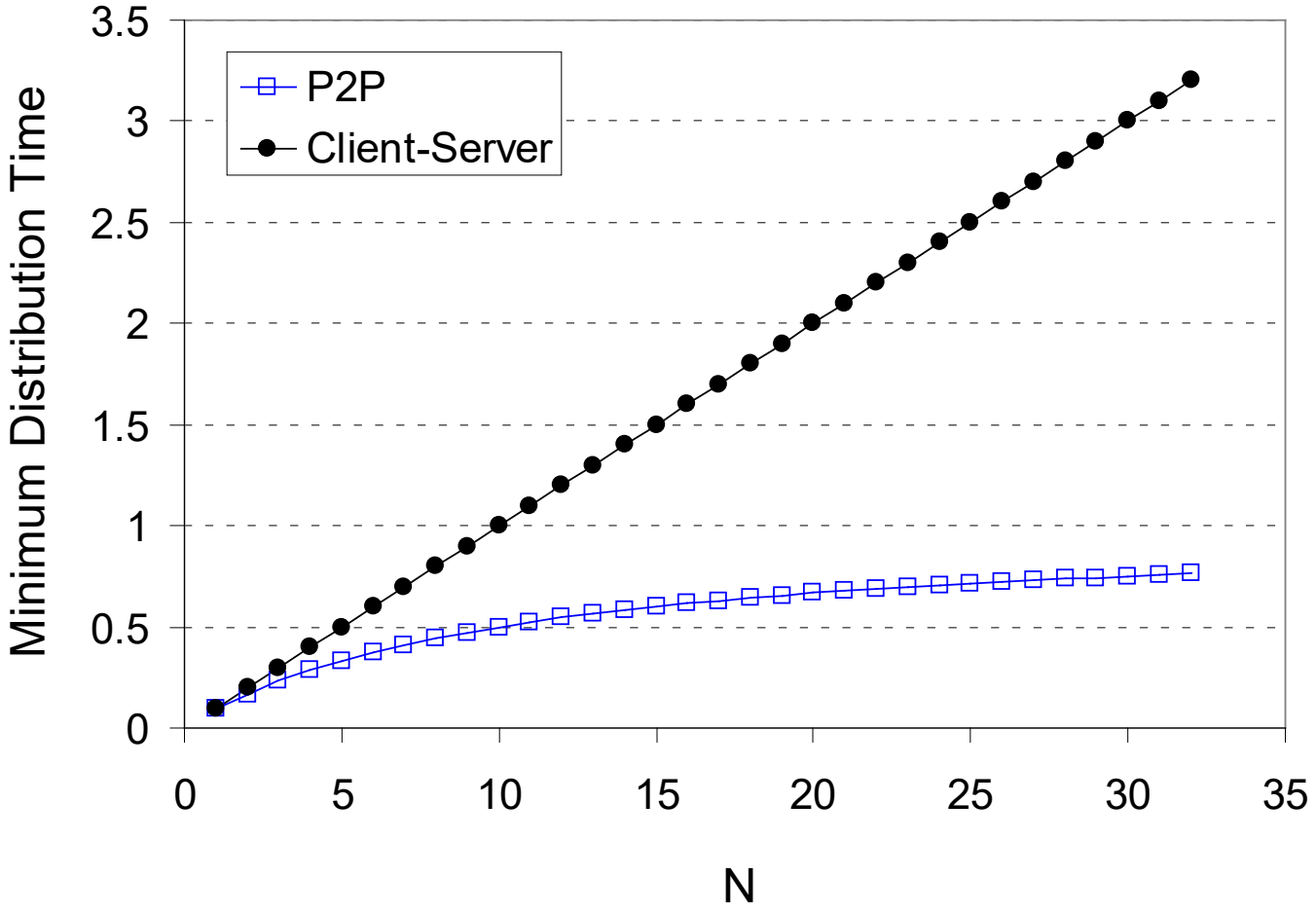


Figure compares the minimum distribution time for the client-server and P2P architectures if all peers have the same upload rate u.

2.8 P2P File Distribution: BitTorrent

- BitTorrent is a popular P2P protocol for file distribution [2011].
- In BitTorrent lingo (tr:dil), the collection of all peers participating in the distribution of a particular file is called a **torrent**.
- Peers in a torrent download equal-size **chunks** of the file from one another, with a typical chunk size of 256 KBytes.

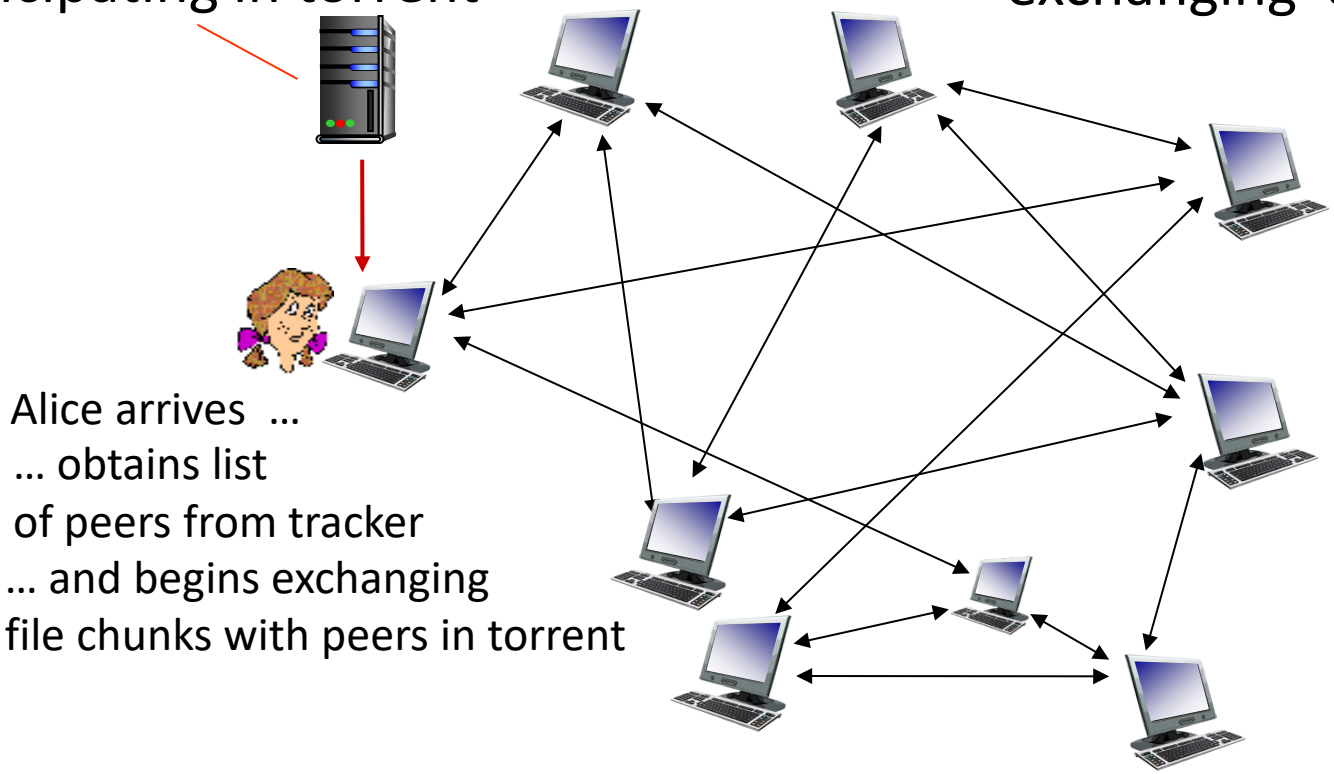
- When a peer first joins a torrent, it has no chunks. Over time it accumulates more chunks. While it downloads chunks it also uploads chunks to other peers.
- Once a peer has acquired the entire file, it may (selfishly) leave the torrent, or (altruistically: tr fedakarca) remain in the torrent and continue to upload chunks to other peers.

2.9 P2P File Distribution: BitTorrent (cont.)

- file divided into 256Kb chunks
- peers in torrent send/receive file chunks

tracker: tracks peers participating in torrent

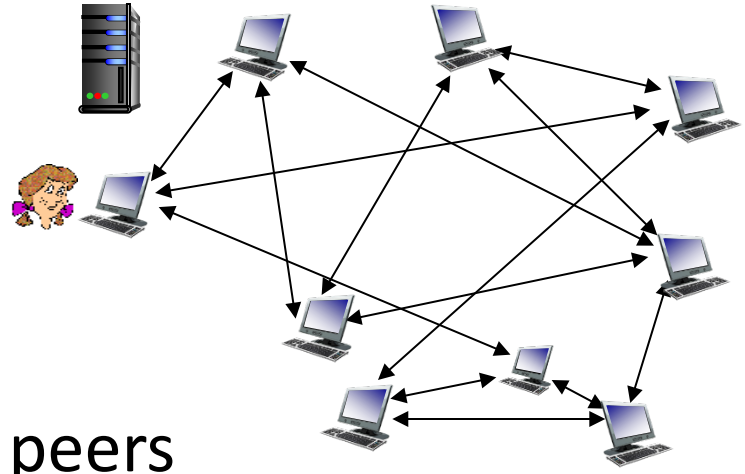
torrent: group of peers exchanging chunks of a file



Alice arrives …
… obtains list
of peers from tracker
… and begins exchanging
file chunks with peers in torrent

2.10 P2P File Distribution: BitTorrent (cont.)

- peer joining torrent:
  - has no chunks, but will accumulate them over time from other peers
  - registers with tracker to get list of peers, connects to subset of peers ("neighbors")
- while downloading, peer uploads chunks to other peers
- peer may change peers with whom it exchanges chunks



- So, at any given instant of time, Alice will have a subset of chunks and will know which chunks her neighbors have. With this information, Alice will have two important decisions to make:
  - First, which chunks should she request first from her neighbors?
  - And second, to which of her neighbors should she send requested chunks?
- First: Alice uses a technique called rarest first. The idea is to determine, from among the chunks she does not have, the chunks that are the rarest among her neighbors.
- Second: BitTorrent uses a clever trading algorithm. The basic idea is that Alice gives priority to the neighbors that are currently supplying her data at the highest rate.

2.11 BitTorrent: Requesting, Sending File Chunks

Requesting chunks:

- at any given time, different peers have different subsets of file chunks
- periodically, Alice asks each peer for list of chunks that they have
- Alice requests missing chunks from peers, rarest first

Sending chunks: tit-for-tat

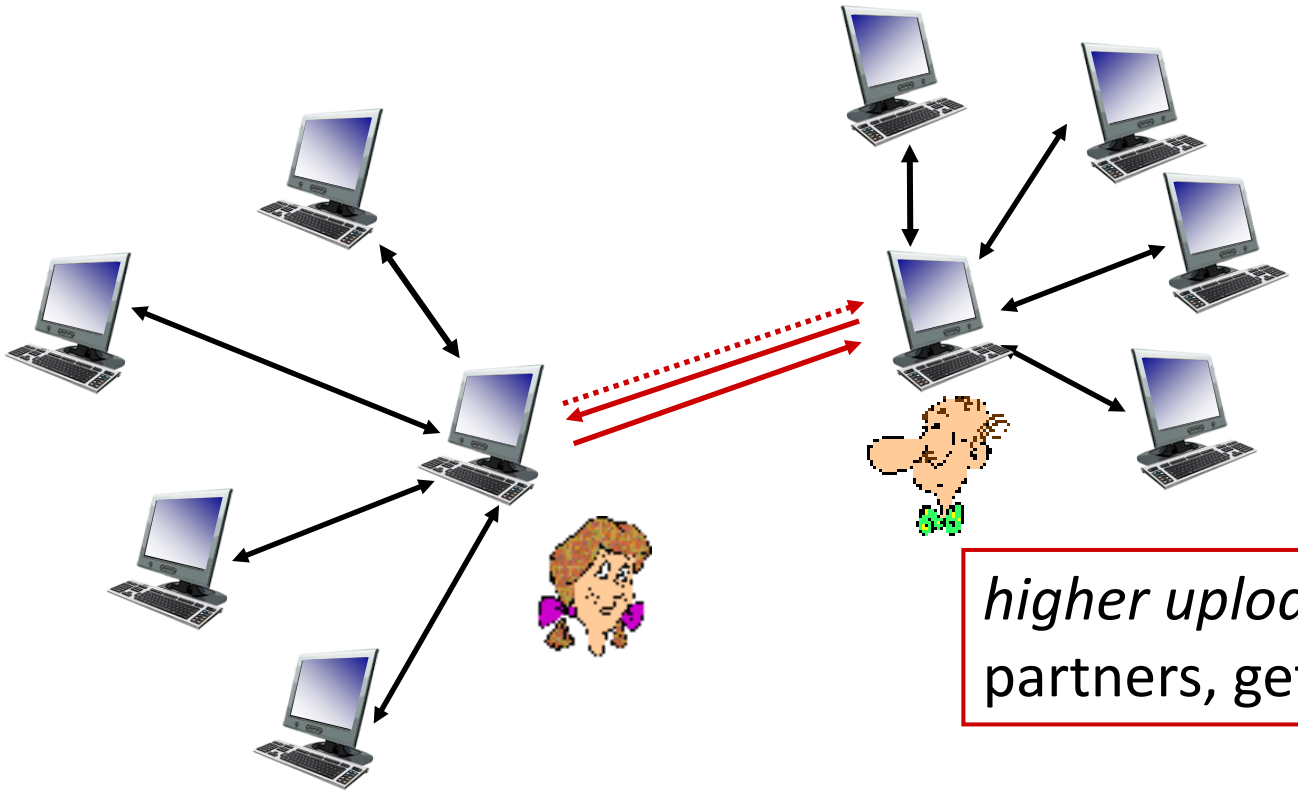
- Alice sends chunks to those four peers currently sending her chunks *at highest rate*
  - other peers are choked by Alice (do not receive chunks from her)
  - re-evaluate top 4 every 10 secs
- every 30 secs: randomly select another peer, starts sending chunks
  - "optimistically unchoke" this peer
  - newly chosen peer may join top 4

2.12 BitTorrent: Tit-for-Tat

(1) Alice “optimistically unchokes” Bob

(2) Alice becomes one of Bob’s top-four providers; Bob reciprocates

(3) Bob becomes one of Alice’s top-four providers



higher upload rate: find better trading partners, get file faster !

2.13 BitTorrent: Two Decisions That Shape Performance

Which chunk first?
Rarest-first selects a missing chunk that few current neighbors have. This helps prevent uncommon chunks from becoming a bottleneck near the end of a swarm.

Upload to whom?
Tit-for-tat gives priority to peers currently providing data at useful rates, encouraging reciprocal contribution.

Why optimistic unchoking?
Occasionally selecting another peer lets a newcomer or a newly useful peer prove that it can contribute.

System effect
Peers exchange chunks while downloading. As a result, the seed does not need to upload every full copy—but peers must remain available and provide useful pieces.

2.14 BitTorrent's Efficiency

- BitTorrent's efficiency is not merely a function of segmenting files into small chunks; it also depends on strategically determining which pieces to request and optimizing the distribution of constrained upload bandwidth…

2.15 Tracker, DHT, and Peer Exchange

Tracker
A coordination service tells a joining peer about other peers participating in a torrent. It generally does not store the actual file chunks.

DHT
A distributed hash table lets peers discover other peers without requiring a single central tracker for every lookup. In BitTorrent, DHT peer discovery uses UDP.

Peer Exchange (PEX)
Connected peers share information about other peers, helping the swarm discover additional participants.

Resilience is not magic
Trackerless discovery reduces dependence on a central tracker, but discovery, connectivity, peer availability, and the original metadata still matter.

2.16 A Real-World Example: A New Peer Joins the Torrent

- Suppose Alice intends to download a Linux distribution via BitTorrent utilizing a magnet link.

1. Acquiring the Torrent Identity
The infohash embedded within the magnet link uniquely identifies the requested torrent. If the required metadata is not yet present, the client may also need to acquire it.

2. Performing Peer Discovery
The client can query a tracker, utilize the Distributed Hash Table (DHT), or evaluate previously discovered peer information.

3. Expanding Discovery via PEX
The client can obtain additional peer information from the peers with which it has established a connection.

4. Downloading the File
Alice retrieves pieces from the connected peers and can simultaneously transmit her own pieces to other peers.

Week 4 - Agenda

02 P2P Applications

03 Video Streaming and Content Distribution Networks

04 Next Week

3.1 Video Streaming and CDNs: Context

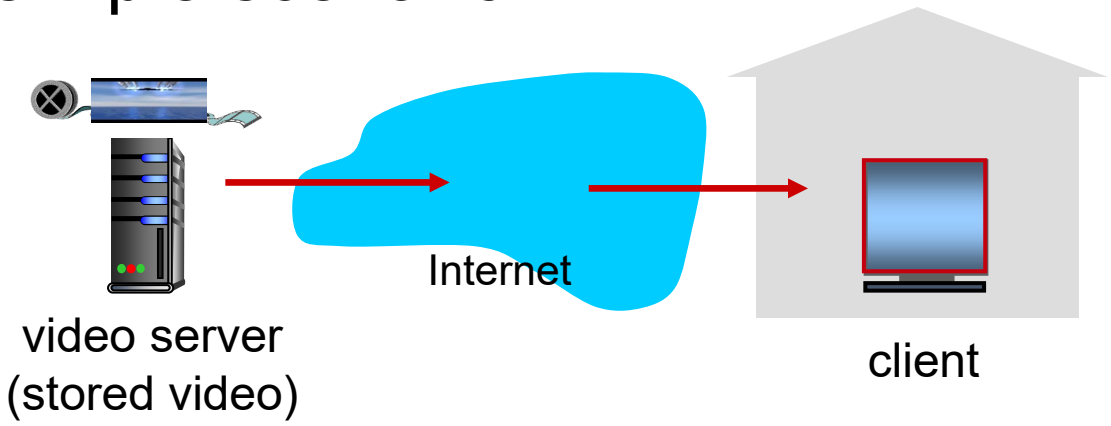
Single 2 Mbps video with a duration of 67 minutes will consume 1 gigabyte of storage and traffic.

Compressed Internet video typically ranges from 100 kbps for low-quality video to over 3 Mbps for streaming high-definition movies; 4K streaming envisions a bitrate of more than 10 Mbps.

- stream video traffic: major consumer of Internet bandwidth
  - Netflix, YouTube, Amazon Prime: 80% of residential ISP traffic (2020)
- *challenge:* scale - how to reach ~1B users?
- *challenge:* heterogeneity
  - different users have different capabilities (e.g., wired versus mobile; bandwidth rich versus bandwidth poor)
- *solution:* distributed, application-level infrastructure

3.2 Streaming Stored Video

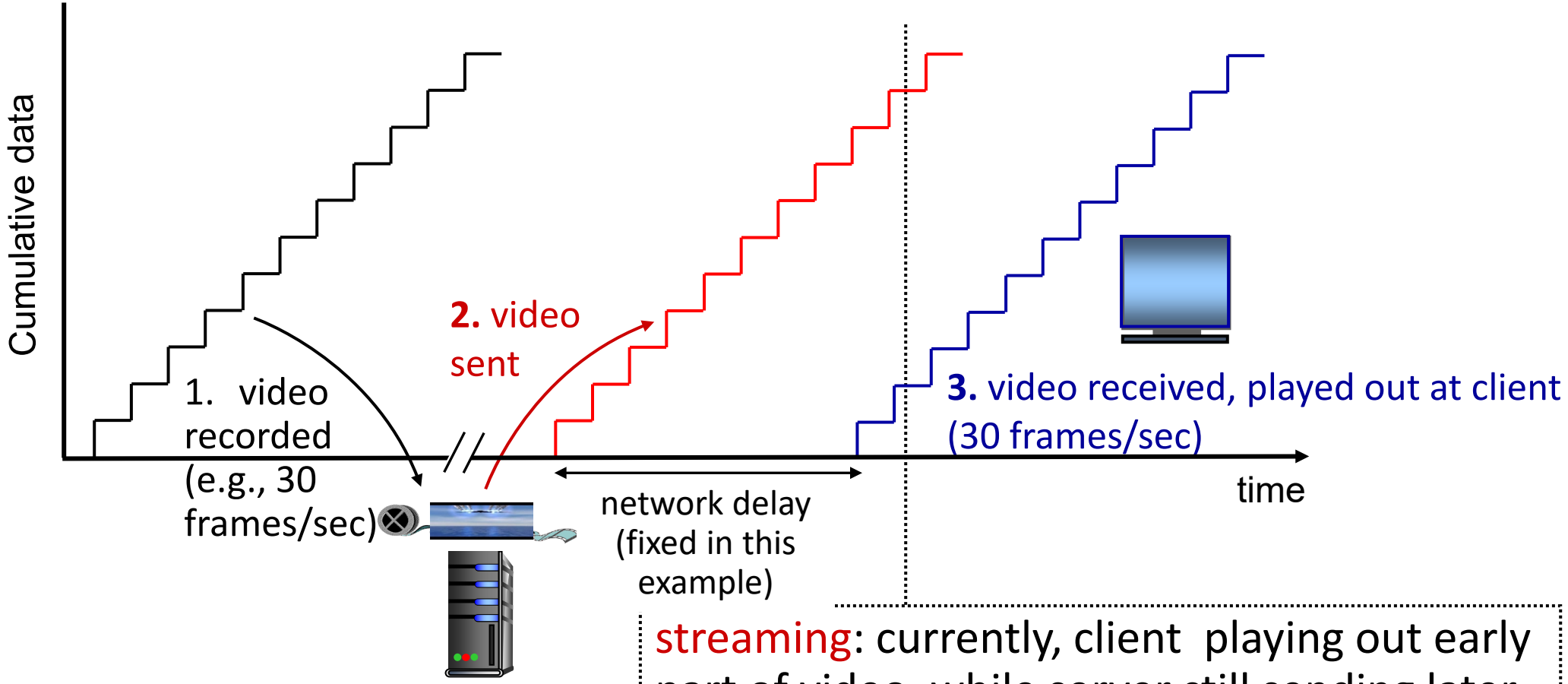
simple scenario:



Main challenges:

- server-to-client bandwidth will *vary* over time, with changing network congestion levels (in house, access network, network core, video server)
- packet loss, delay due to congestion will delay playout, or result in poor video quality

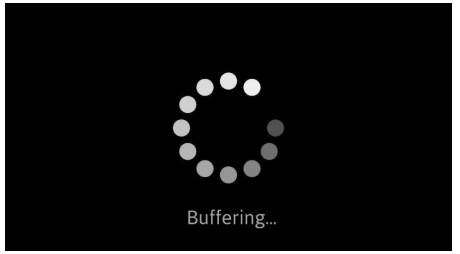
3.3 Streaming Stored Video (cont.)



streaming: currently, client playing out early part of video, while server still sending later part of video

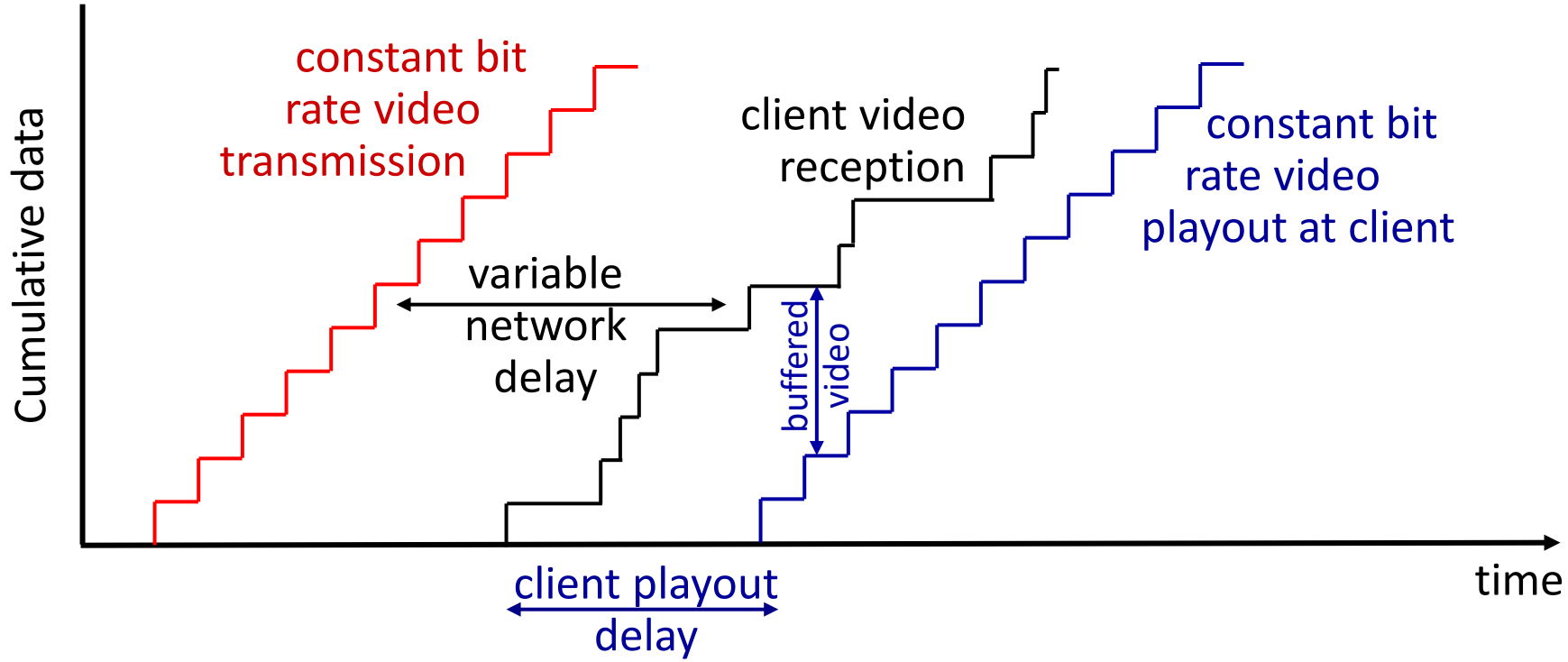
3.4 Streaming Stored Video: Challenges

- continuous playout constraint: during client video playout, playout timing must match original timing
  - … but network delays are variable (jitter), so will need client-side buffer to match continuous playout constraint



- other challenges:
  - client interactivity: pause, fast-forward, rewind, jump through video
  - video packets may be lost, retransmitted

3.5 Streaming Stored Video: Playout Buffering



- *client-side buffering and playout delay:* compensate for network-added delay, delay jitter

3.6 Why Buffering Happens: Throughput vs Playback Rate

Playback rate
The client consumes buffered media at the encoded playback rate (for example, 4 Mbit/s).

Network throughput
If the connection delivers chunks faster than playback consumes them, buffer grows; if slower for long enough, buffer shrinks.

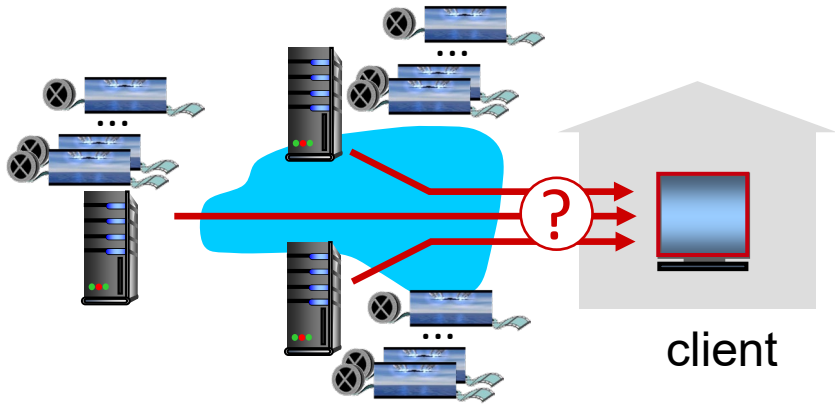
Jitter and outages
Average throughput can look acceptable while short dips, retransmissions, congestion, or Wi-Fi interference still drain the buffer.

Buffer trade-off
More buffering can reduce stalls but increases startup delay and may make live interaction less responsive.

3.7 Streaming Multimedia: DASH (Dynamic Adaptive Streaming over HTTP)

server:

- divides video file into multiple chunks
- each chunk encoded at multiple different rates
- different rate encodings stored in different files
- files replicated in various CDN nodes
- *manifest file:* provides URLs for different chunks

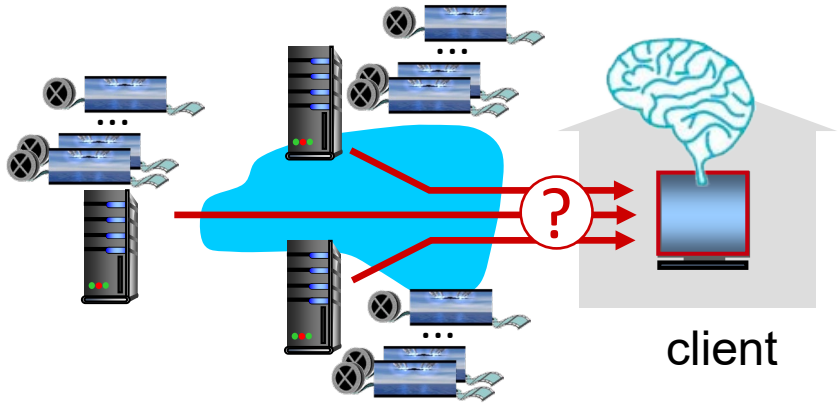


client:

- periodically estimates server-to-client bandwidth
- consulting manifest, requests one chunk at a time
  - chooses maximum coding rate sustainable given current bandwidth
  - can choose different coding rates at different points in time (depending on available bandwidth at time), and from different servers

3.8 Streaming Multimedia: DASH

- *"intelligence"* at client: client determines
  - *when* to request chunk (so that buffer starvation, or overflow does not occur)
  - *what encoding rate* to request (higher quality when more bandwidth available)
  - *where* to request chunk (can request from URL server that is "close" to client or has high available bandwidth)



Streaming video = encoding + DASH + playout buffering

3.9 Content Distribution Networks (CDNs)

challenge: how to stream content (selected from millions of videos) to hundreds of thousands of *simultaneous* users?

- *option 1:* single, large "mega-server"
  - single point of failure
  - point of network congestion
  - long (and possibly congested) path to distant clients

….quite simply: this solution *doesn't scale*

3.10 Content Distribution Networks (cont.)

challenge: how to stream content (selected from millions of videos) to hundreds of thousands of *simultaneous* users?

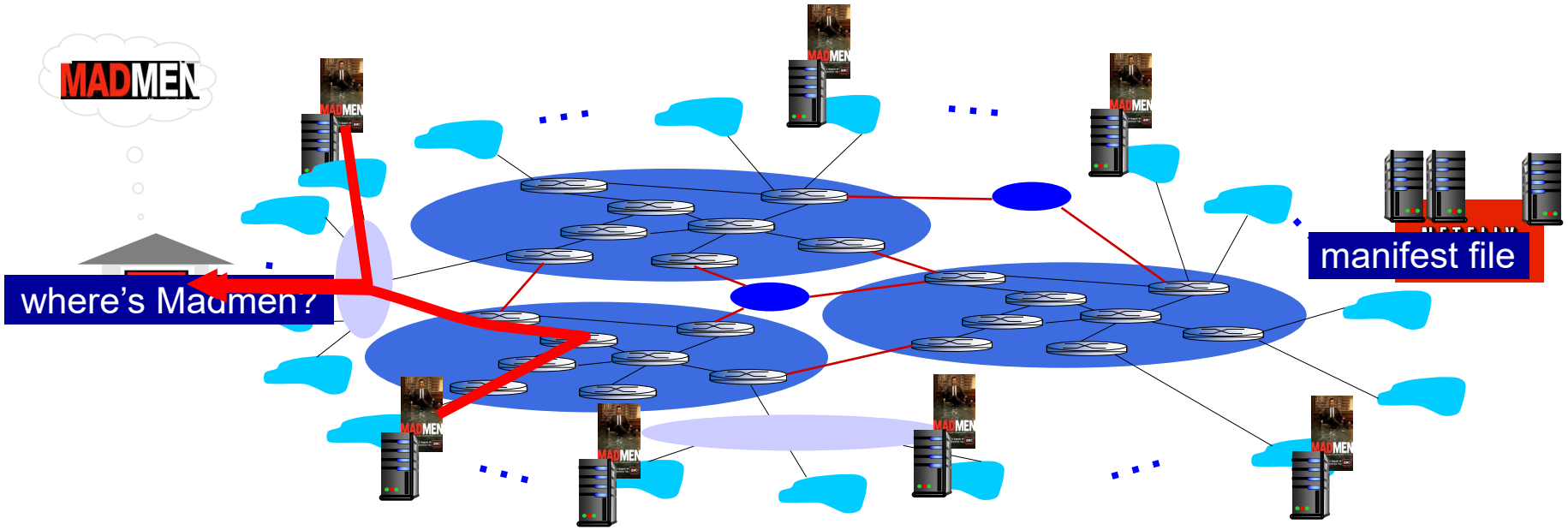
- *option 2:* store/serve multiple copies of videos at multiple geographically distributed sites *(CDN)*
  - *enter deep:* push CDN servers deep into many access networks
    - close to users
    - Akamai: 240,000 servers deployed in > 120 countries (2015)
  - *bring home:* smaller number (10's) of larger clusters in POPs near access nets
    - used by Limelight





3.11 Content Distribution Networks (cont.)

- CDN: stores copies of content (e.g. MADMEN) at CDN nodes
- subscriber requests content, service provider returns manifest
  - using manifest, client retrieves content at highest supportable rate
  - may choose different rate or copy if network path congested



3.12 Case Study: Netflix Open Connect

Deployment model
Netflix describes Open Connect as an ISP partnership program combining embedded Open Connect Appliances and interconnection/peering.

Why embed appliances?
Frequently watched content can be served inside or near an ISP network, reducing reliance on long transit paths and helping localize traffic.

Control + data roles
A control system and intelligent clients help choose where content is fetched; appliances serve video objects close to subscribers.

Transferable lesson
Large-scale delivery is a systems problem involving cache placement, demand prediction, ISP relationships, health monitoring, and routing—not just CPU speed.

4.1 Next Week

- We will focus on chapters 3: Transport Layer.
- We will also focus on Wireshark usage and examples.



Your duties:

✓ Review this lecture note via learn.khas.edu.tr

✓ Read chapter 3 from Kurose & Ross's textbook.

Thank You
For Your Attention!

